



WIF3009

Python for Scientific Computing

Applied Deep Learning (Part 1)



Intro to Deep Learning

Introduction to Deep Learning(what is ?)

- Traditional model of pattern recognition (since the late 50s)
 - Fixed/engineered features (or fixed kernel)
 - Trainable “simple” classifier

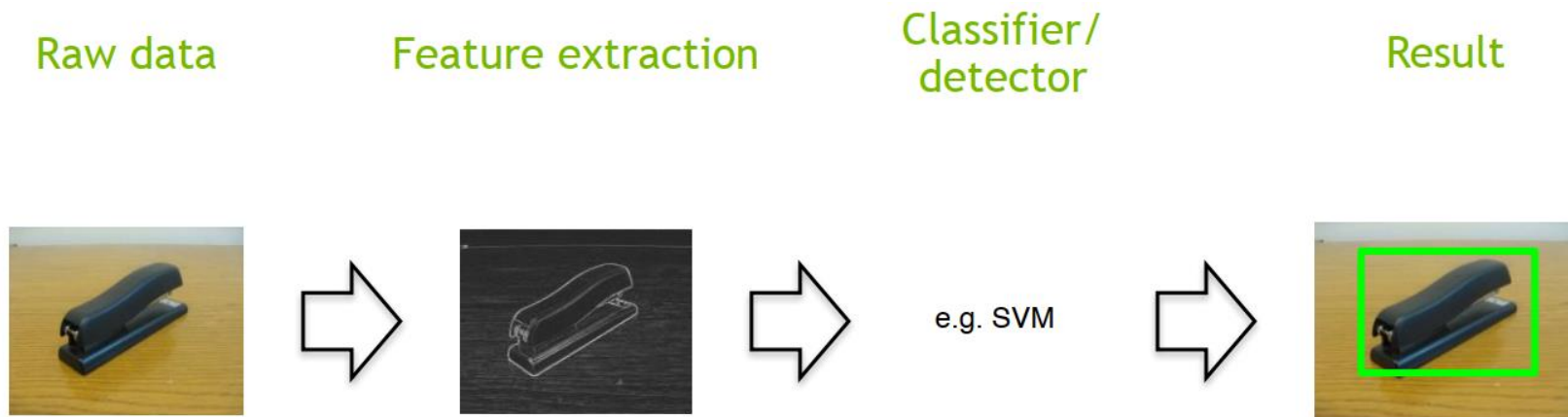


Hand-crafted
Feature Extractor

“Simple” Trainable
Classifier

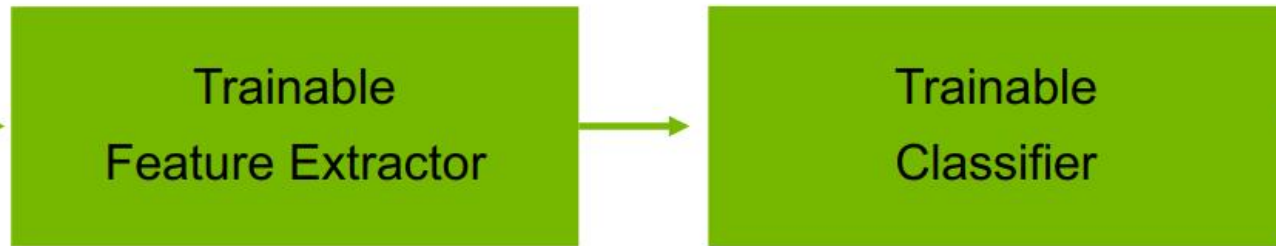
Introduction to Deep Learning(what is ?)

- Traditional machine perception



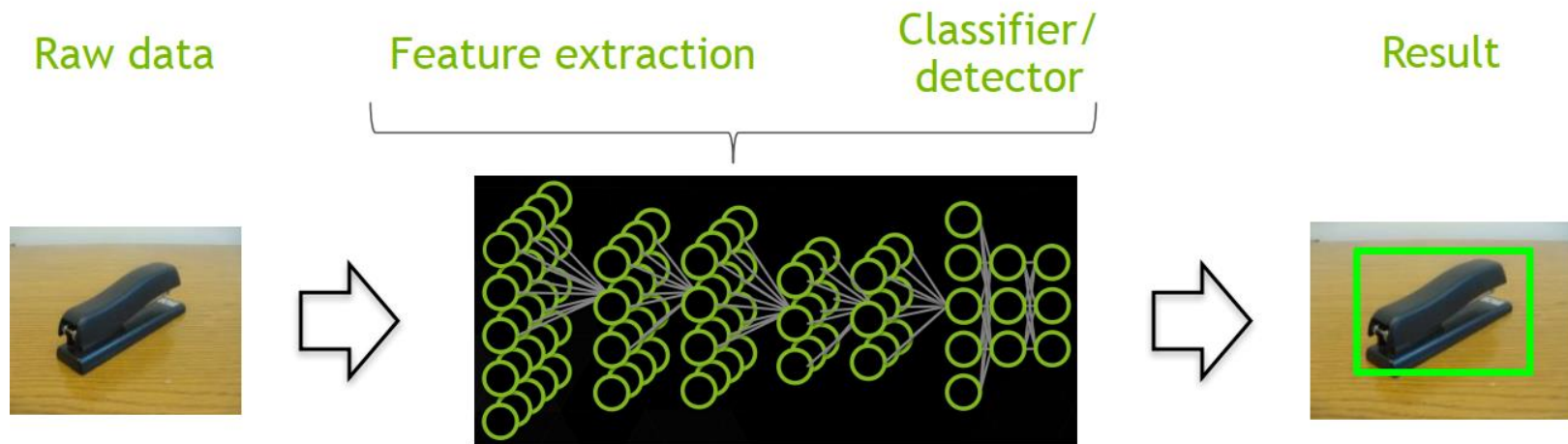
Introduction to Deep Learning(what is ?)

- End-to-end learning / Feature learning / Deep learning
 - Trainable features (or trainable kernel)
 - Trainable classifier



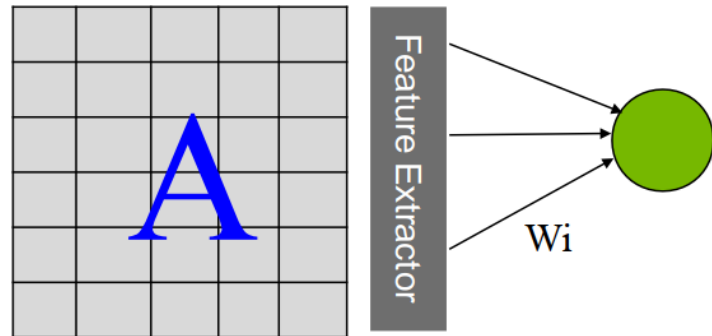
Introduction to Deep Learning(what is ?)

- Deep learning machine perception



Introduction to Deep Learning (background)

- The basic model has not evolved much since the 50's
- The first learning machine: **The Perceptron**
- The Perceptron:
 - A simple **linear classifier** on top of a simple feature extractor
- The vast majority of practical applications in ML today consist of a glorified **linear classifiers**.



Introduction to Deep Learning (Inference)

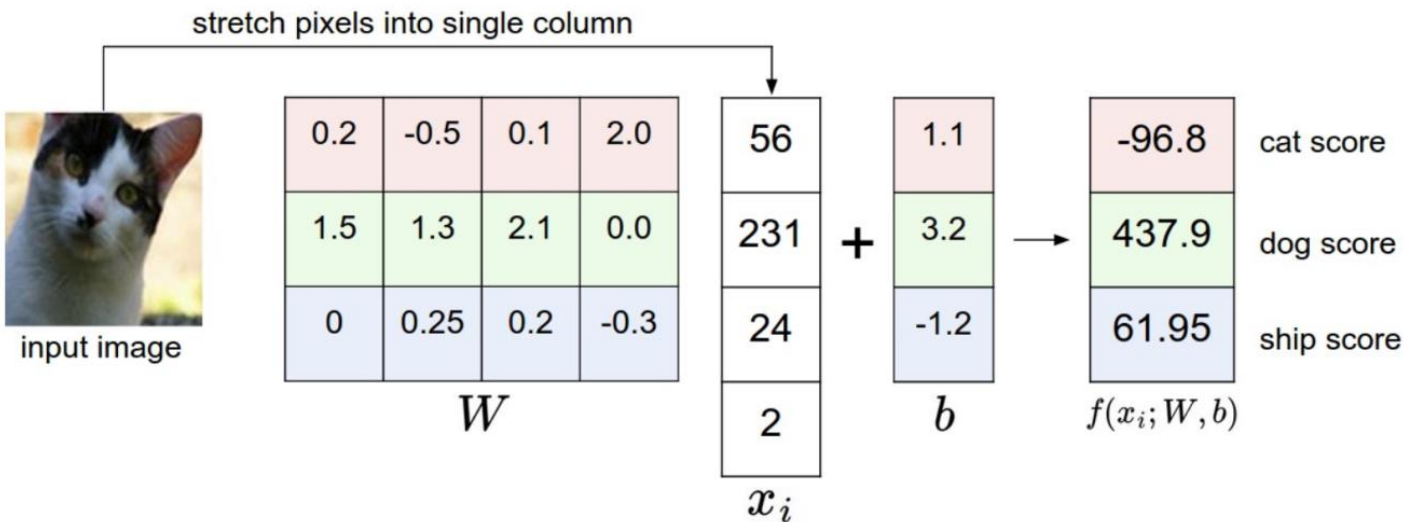
The diagram illustrates the inference equation $f(x_i, W, b) = Wx_i + b$. It features several color-coded annotations with arrows pointing to specific parts of the equation:

- Class Score:** A green arrow points from the text "Class Score" to the function $f(x_i, W, b)$.
- Input Data:** A purple arrow points from the text "Input Data" to the variable x_i .
- Weights:** A red arrow points from the text "Weights" to the variable W .
- Bias:** A red arrow points from the text "Bias" to the variable b .
- "Parameters":** The text "Parameters" is written in red below the variables W and b , indicating they are the model's parameters.

Introduction to Deep Learning (inference)

- A simple linear classifier for image:

$$f(x_i, W, b) = Wx_i + b$$



Introduction to Deep Learning (Loss function)



- Also known as a cost function or objective function
- It quantifies how well a model's predictions match the actual target values (**ground truth**) for a given dataset.
- The goal of training a machine learning model is to **minimize this loss function.**
- As a **lower loss** indicates that the model is **making more accurate predictions.**
- The choice of an appropriate loss function depends on the specific task and the type of data (e.g., regression, classification, or other machine learning tasks).

Introduction to Deep Learning (Loss function)



We want to define a loss function L_i that describes how unhappy we are with these scores.

Introduction to Deep Learning (Loss function)

- Common loss function for regression tasks:

- Mean absolute error (MAE)

$$MAE = \frac{1}{n} \sum_{j=1}^n |y_j - \hat{y}_j|$$

- Mean squared error (MSE)

$$MSE = \frac{1}{n} \sum \underbrace{(y - \hat{y})^2}_{\substack{\text{The square of the difference} \\ \text{between actual and} \\ \text{predicted}}}$$

- Root mean squared error (RMSE)

$$RMSE = \sqrt{\frac{\sum (y_i - \hat{y}_i)^2}{N - P}}$$

Introduction to Deep Learning (Loss function)

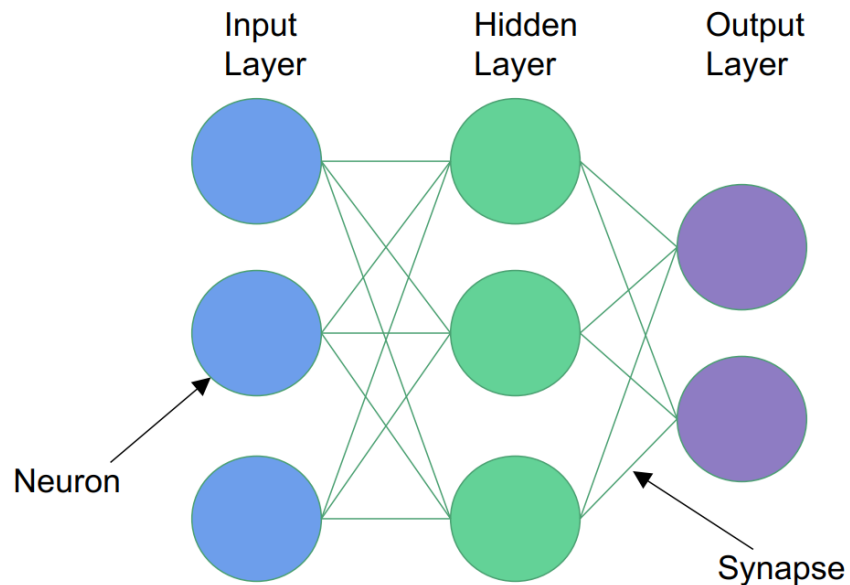


- Common loss function for classification tasks:
- Cross-entropy loss
 - Commonly used in classification tasks, particularly for problems involving binary or multiclass classification.
 - It measures the **dissimilarity** between the predicted class probabilities and the true class labels.

$$\text{Loss} = - \sum_{i=1}^{\text{output size}} y_i \cdot \log \hat{y}_i$$

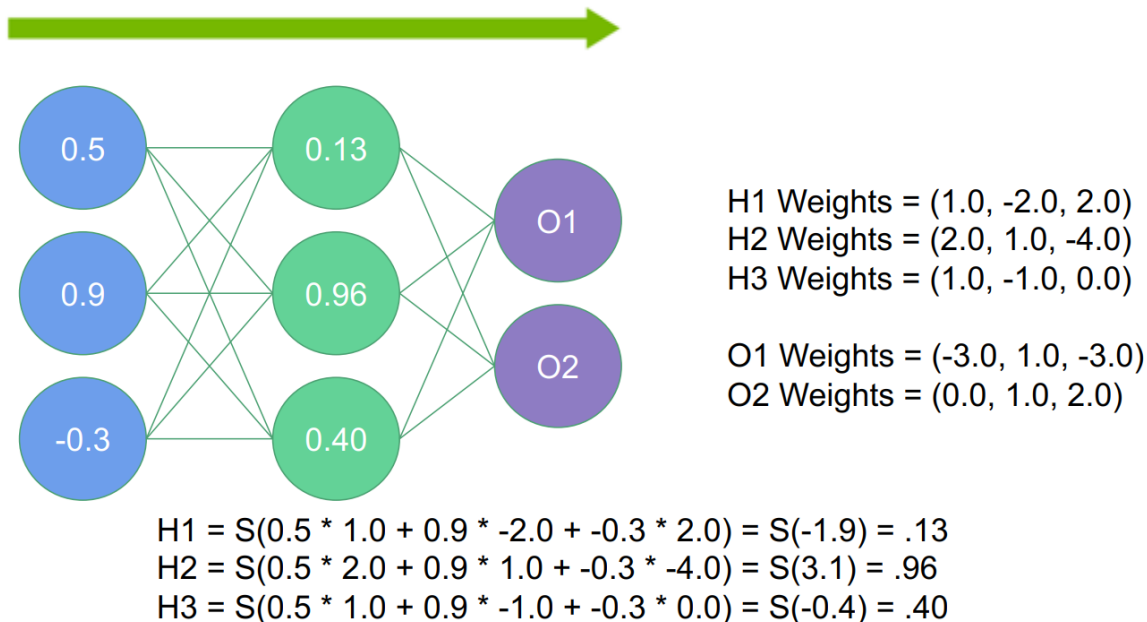
Introduction to DL (Neural Network Architecture)

- Artificial Neural Network (ANN)



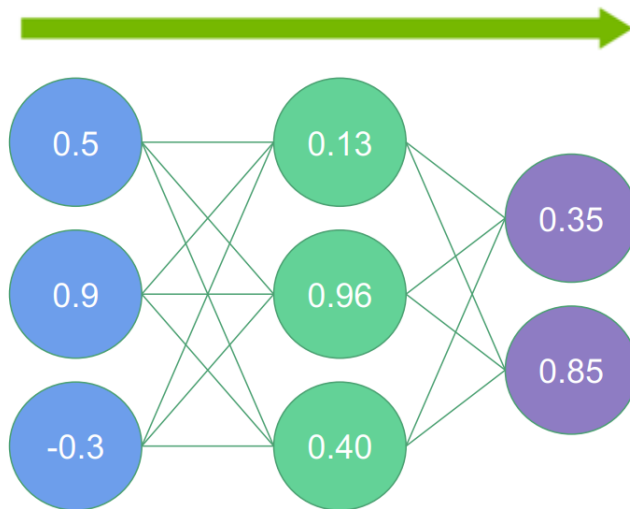
Introduction to DL (Neural Network Architecture)

- Inference/Feed-forward:



Introduction to DL (Neural Network Architecture)

- Inference/Feed-forward:



H1 Weights = (1.0, -2.0, 2.0)

H2 Weights = (2.0, 1.0, -4.0)

H3 Weights = (1.0, -1.0, 0.0)

O1 Weights = (-3.0, 1.0, -3.0)

O2 Weights = (0.0, 1.0, 2.0)

$$O1 = S(.13 * -3.0 + .96 * 1.0 + .40 * -3.0) = S(-.63) = .35$$

$$O2 = S(.13 * 0.0 + .96 * 1.0 + .40 * 2.0) = S(1.76) = .85$$

Introduction to DL (Neural Network Architecture)

- Matrix formulation:

H1 Weights = (1.0, -2.0, 2.0)

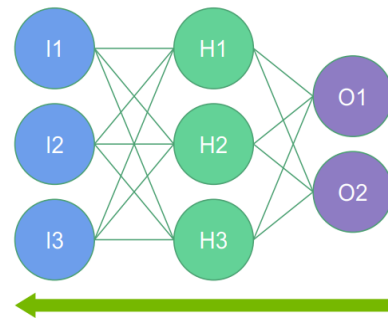
H2 Weights = (2.0, 1.0, -4.0)

H3 Weights = (1.0, -1.0, 0.0)

$$S\left(\begin{array}{|c|c|c|} \hline \text{Hidden Layer Weights} & & \\ \hline 1.0 & -2.0 & 2.0 \\ \hline 2.0 & 1.0 & -4.0 \\ \hline 1.0 & -1.0 & 0.0 \\ \hline \end{array}\right) * \begin{array}{|c|} \hline \text{Inputs} \\ \hline 0.5 \\ \hline 0.9 \\ \hline -0.3 \\ \hline \end{array} = S\left(\begin{array}{|c|c|c|} \hline -1.9 & 3.1 & -0.4 \\ \hline \end{array}\right) = \begin{array}{|c|c|c|} \hline \text{Hidden Layer Outputs} \\ \hline .13 & .96 & 0.4 \\ \hline \end{array}$$

Introduction to DL (Backpropagation)

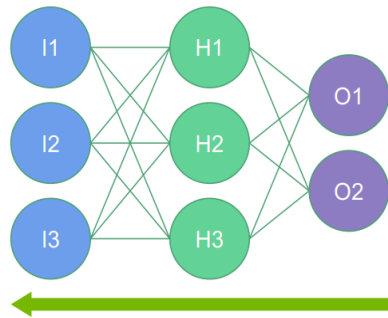
- Problem: Which weights should be updated and by how much?
- Enter: **BACKPROPAGATION!**
- Short for "**backward propagation of errors**"
- Fundamental algorithm used in training artificial neural networks, including deep learning models.
- Usage:
 - To **adjust the model's weights and biases** during training
 - To **minimize the error** between the predicted output and the actual target values.



backward to input to update parameters with loss

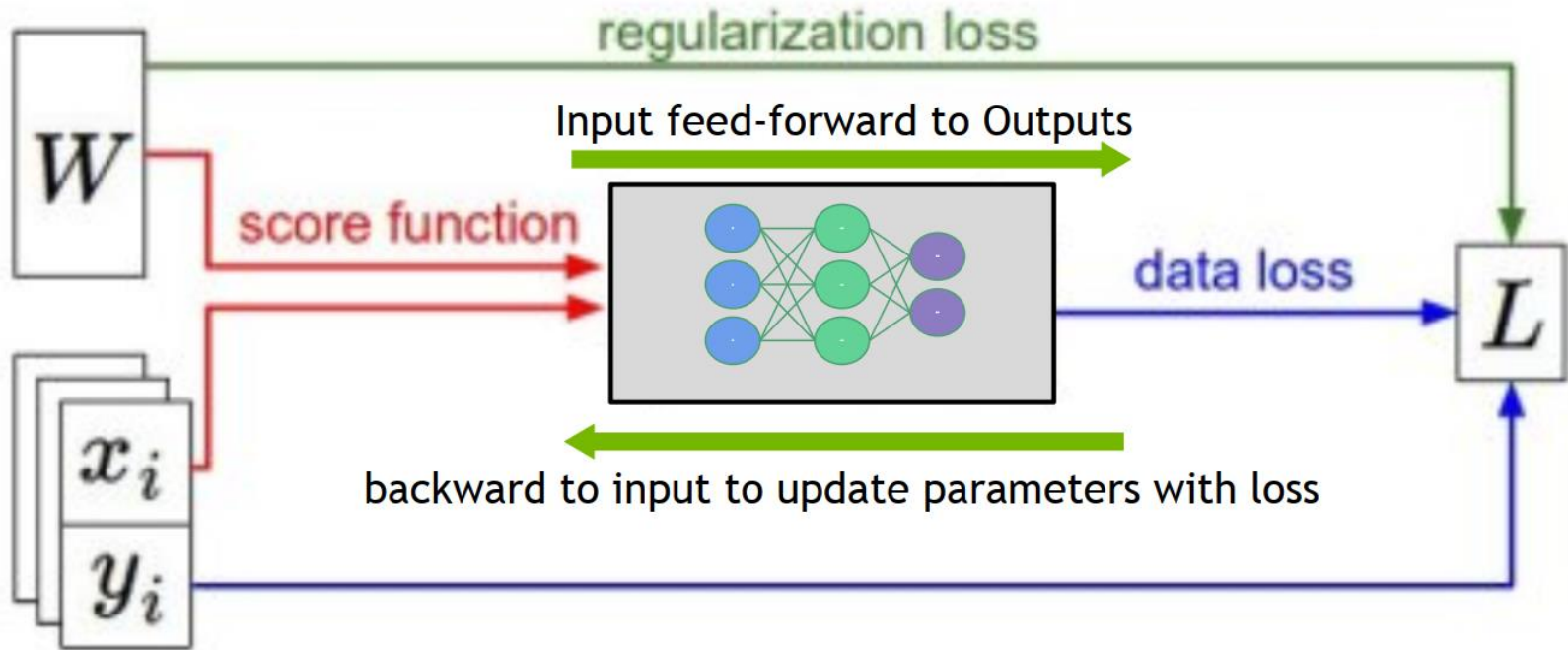
Introduction to DL (Backpropagation)

- Gradient descent is a fundamental optimization algorithm used in machine learning and deep learning to **minimize a cost or loss function**.
- The **primary goal** of gradient descent is to find the values of model parameters (such as weights and biases) that **minimize the cost function!**
- Thereby making the model's predictions as **accurate as possible**.
- How?
 - At each time step, the error of the network is calculated on the training data
 - Then, the weights are modified to reduce the error



backward to input to update parameters with loss

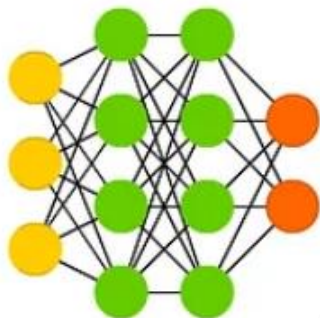
Introduction to DL (Summary)



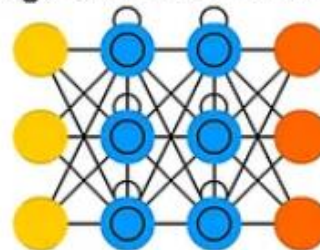
Introduction to DL (Summary)

- Backfed Input Cell
- Input Cell
- Noisy Input Cell
- Hidden Cell
- Probablistic Hidden Cell
- Spiking Hidden Cell
- Output Cell
- Match Input Output Cell
- Recurrent Cell
- Memory Cell
- Different Memory Cell
- Kernel
- Convolution or Pool

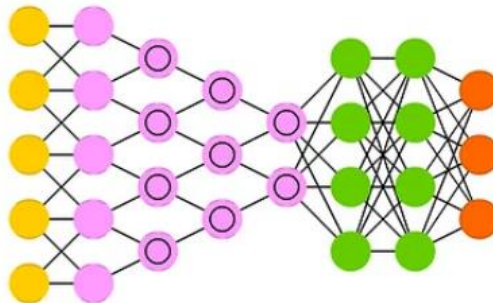
Deep Feed Forward (DFF)



Long / Short Term Memory (LSTM)



Deep Convolutional Network (DCN)

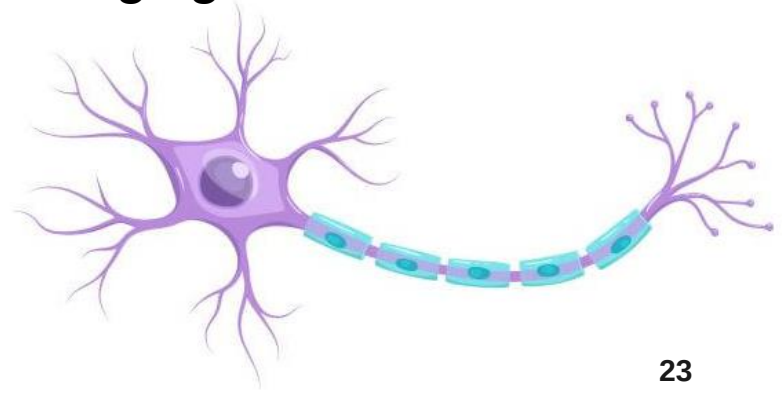




Architecture: Artificial Neural Network (ANN)

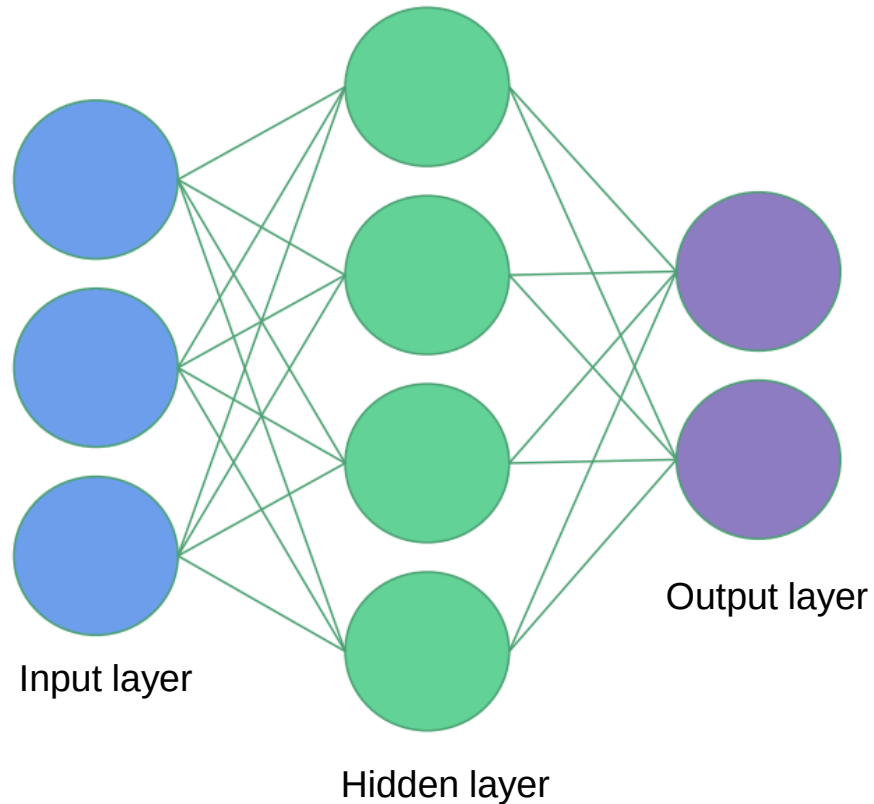
ANN (Introduction)

- An **Artificial Neural Network** (ANN), often simply referred to as a neural network.
- A computational model inspired by the structure and function of the **human brain's biological** neural networks.
- Neural networks are a **subset of machine learning algorithms** and are particularly well-suited for tasks involving:
 - Pattern recognition
 - Classification
 - Regression
 - other complex data-driven tasks.



ANN (Introduction)

- A simple ANN
 - 2 layer NN
 - Or
 - 1 hidden layer NN



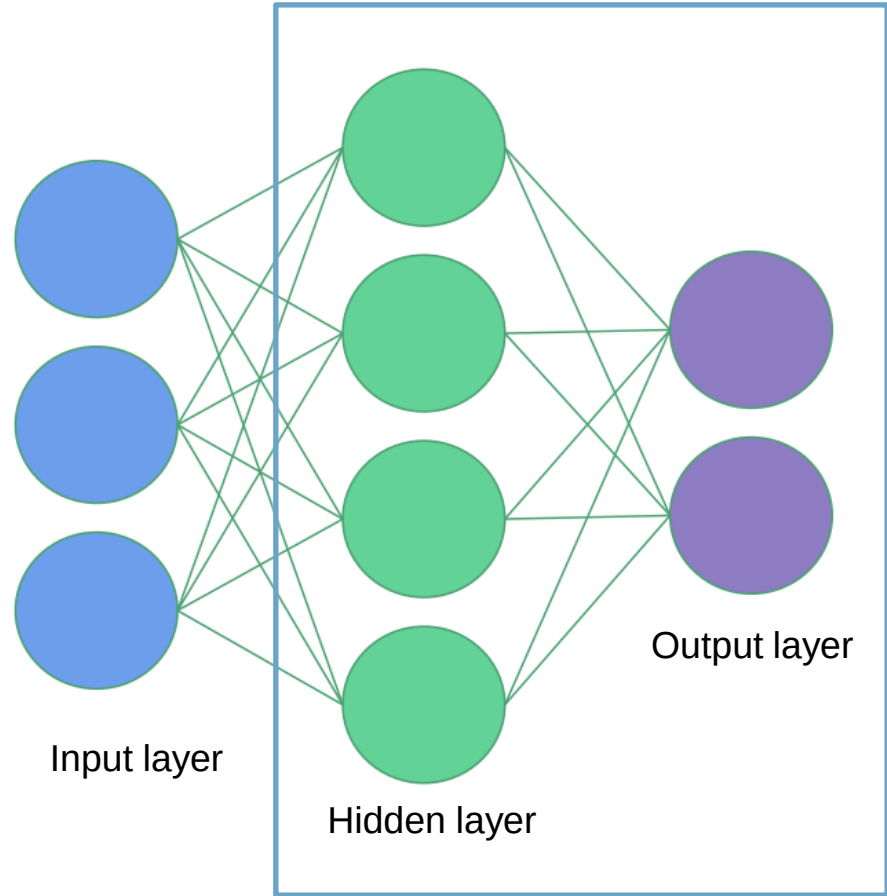
ANN (Introduction)

- Trainable parameters:

Number of Neurons: $4 + 2 = 6$

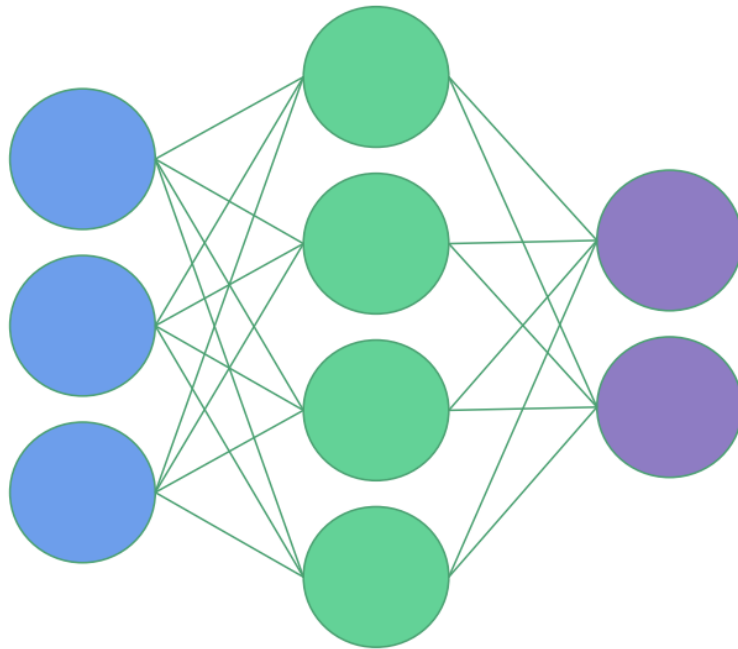
Number of Weights: $[4 \times 3 + 2 \times 4] = 20$

Number of Parameters: $6 + 20 = 26$



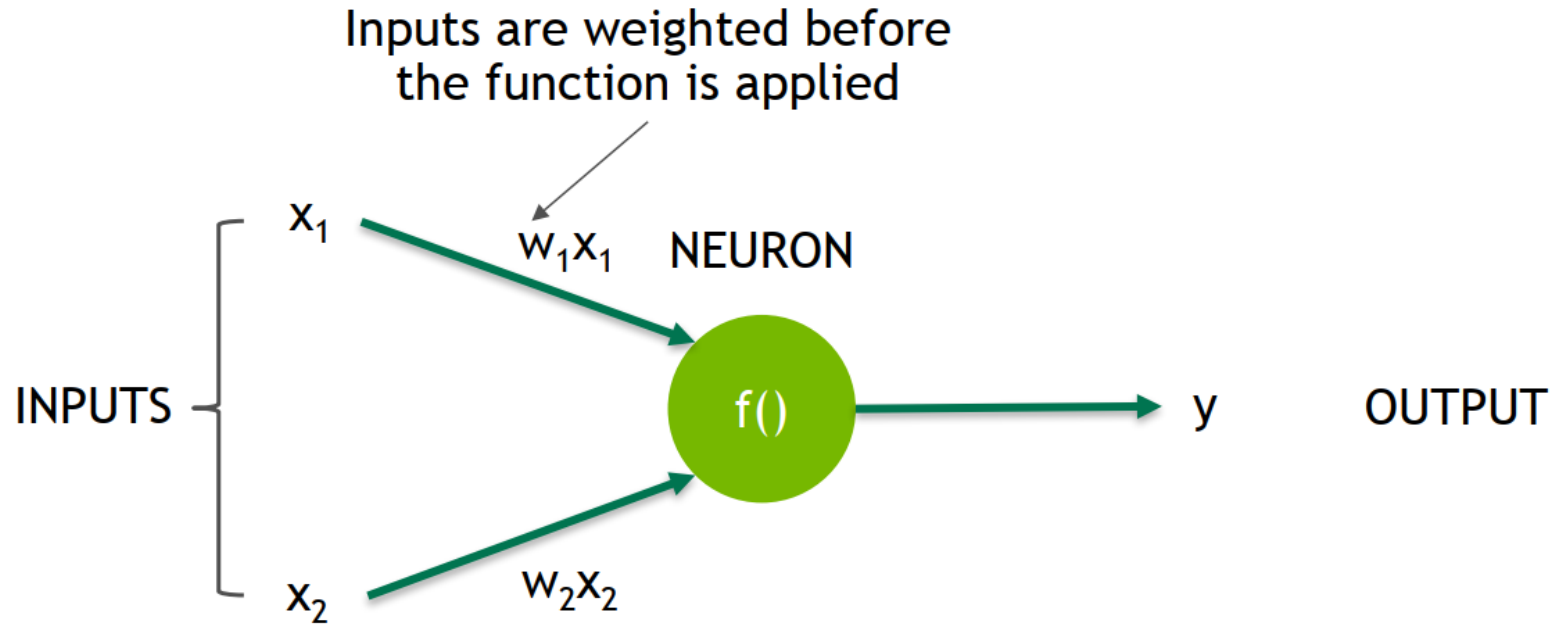
ANN (Introduction)

- Trainable parameters:



Modern CNNs: ~10 million neurons
Human visual cortex: ~5 billion neurons

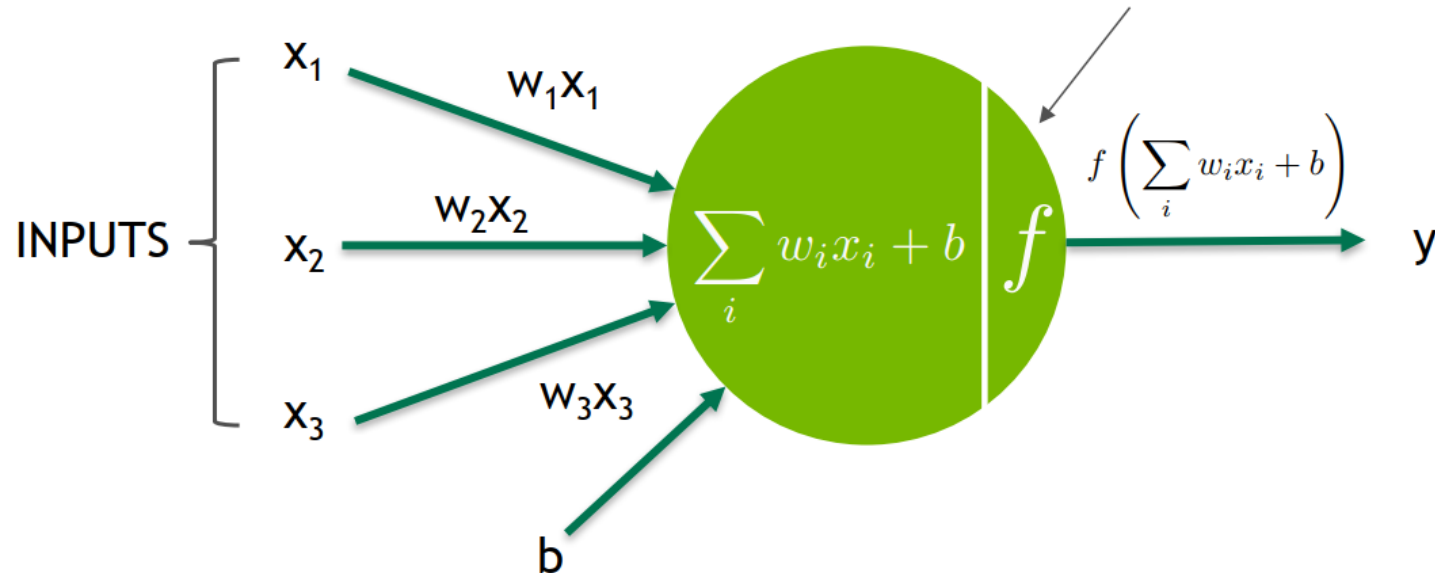
ANN (The Artificial Neuron)



ANN (The Artificial Neuron)

More detailed view

Activation function f

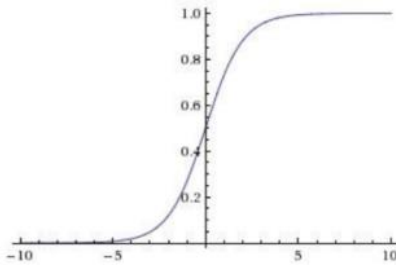


ANN (Activation Function)

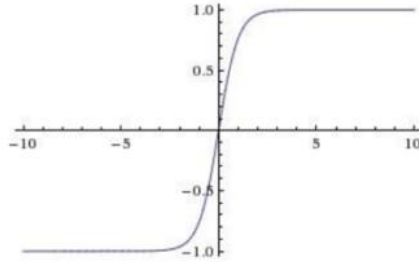


- It is applied to the weighted sum of inputs to a neuron or node to determine its output.
- Activation functions **introduce non-linearity** into the model, allowing it to **learn complex patterns** and relationships in the data.
- Non-linearity is important because a multi-layer ANN with **linear activations** would still only be able to learn composite linear functions.

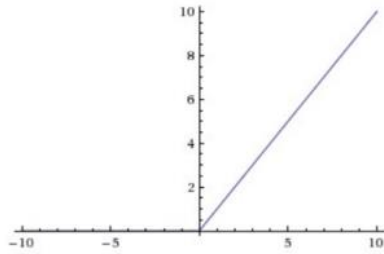
ANN (Activation Function)



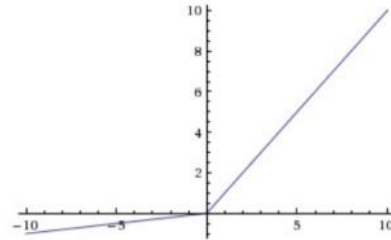
Sigmoid



$\tanh(x)$

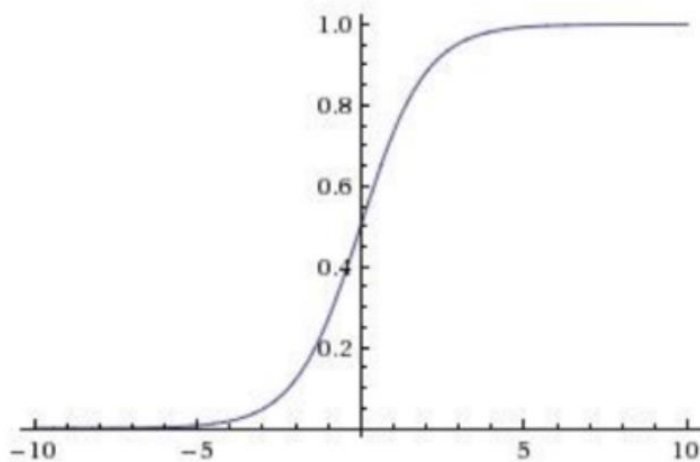


ReLU



Leaky ReLU

ANN (Activation Function - Sigmoid)

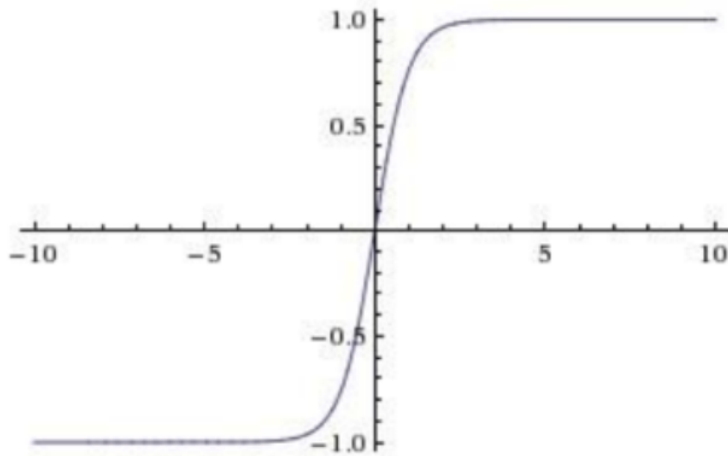


Sigmoid

$$\sigma(x) = 1/(1 + e^{-x})$$

- ▶ Squashes numbers to the range $[0,1]$
- ▶ Historically popular, but they have big practical problems that prevent network training

ANN (Activation Function – Hyperbolic Tangent)

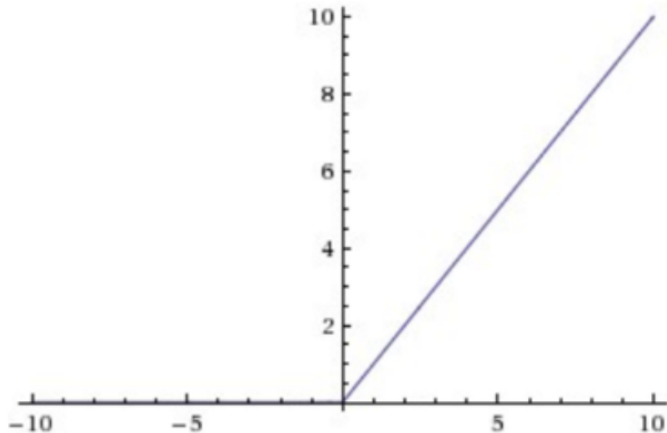


$\tanh(x)$

- ▶ Squashes numbers to range $[-1,1]$
 - ▶ Zero-centered is good
- ▶ Still have practical issues

ANN (Activation Function – Rectifier Linear Units)

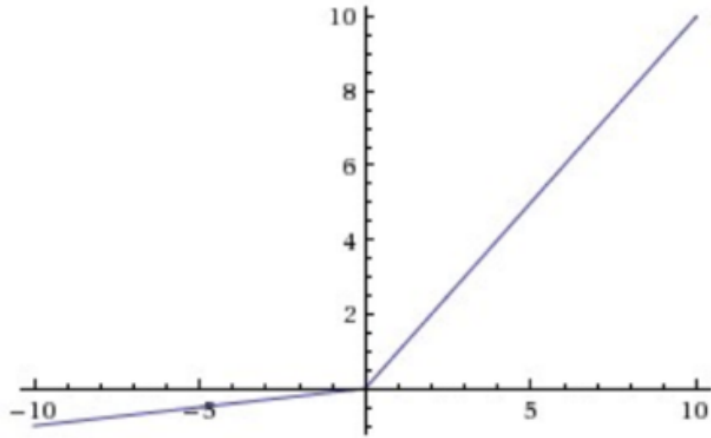
$$f(x) = \max(0, x)$$



ReLU

- ▶ Most popular in current DL work
- ▶ Computationally efficient, ReLU neurons can be trained much faster
- ▶ Addresses the practical issues of sigmoid and tanh
- ▶ Neurons can “die” during training

ANN (Activation Function – Leaky ReLU)



- ▶ Computationally efficient
- ▶ Will not “die” like ReLU

Leaky ReLU

ANN (Activation Function – What to use in practice?)



- In practice, what activation function you should use?
- **Use ReLU**
- Try out Leaky ReLU
- You can try out tanh but it probably won't work as well as ReLU
- **Avoid to use Sigmoid if can!**

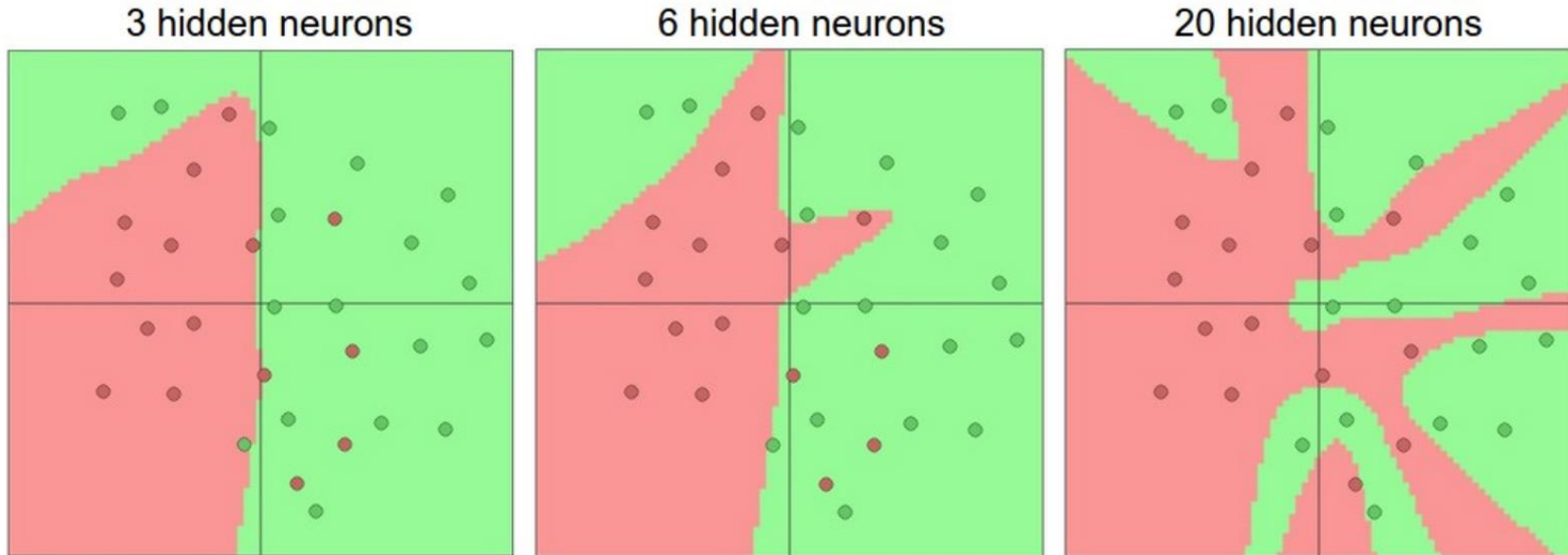
ANN (Training a neural network)



- Procedure for training Neural Networks:
 - Perform inference/forward-pass on the training set.
 - Calculate the error between the predictions and actual labels of the training set.
 - Determine the contribution of each Neuron to the error.
 - Modify the weights of the Neural Network to minimize the error.
- Error contributions are calculated using Backpropagation.
- Error minimization is achieved with Gradient Descent

ANN (Setting the number of layers & size)

Overfitting, generalization and outliers

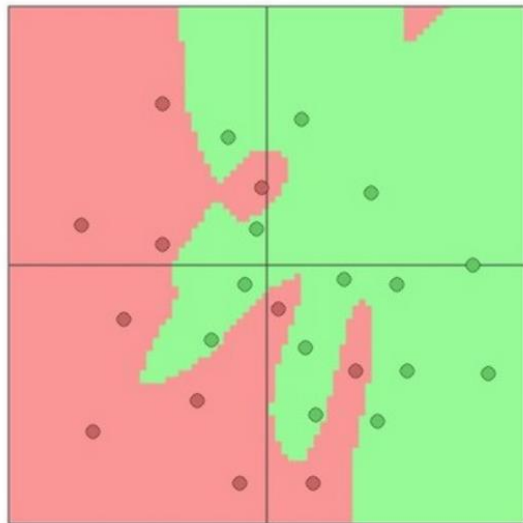


ANN (Effect of regularization)

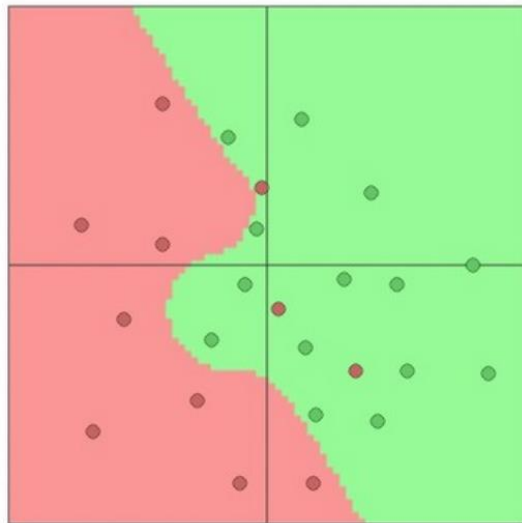


Overfitting, generalization and outliers

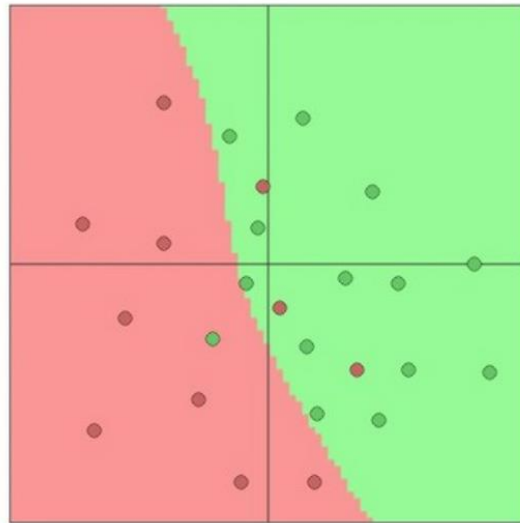
$\lambda = 0.001$



$\lambda = 0.01$

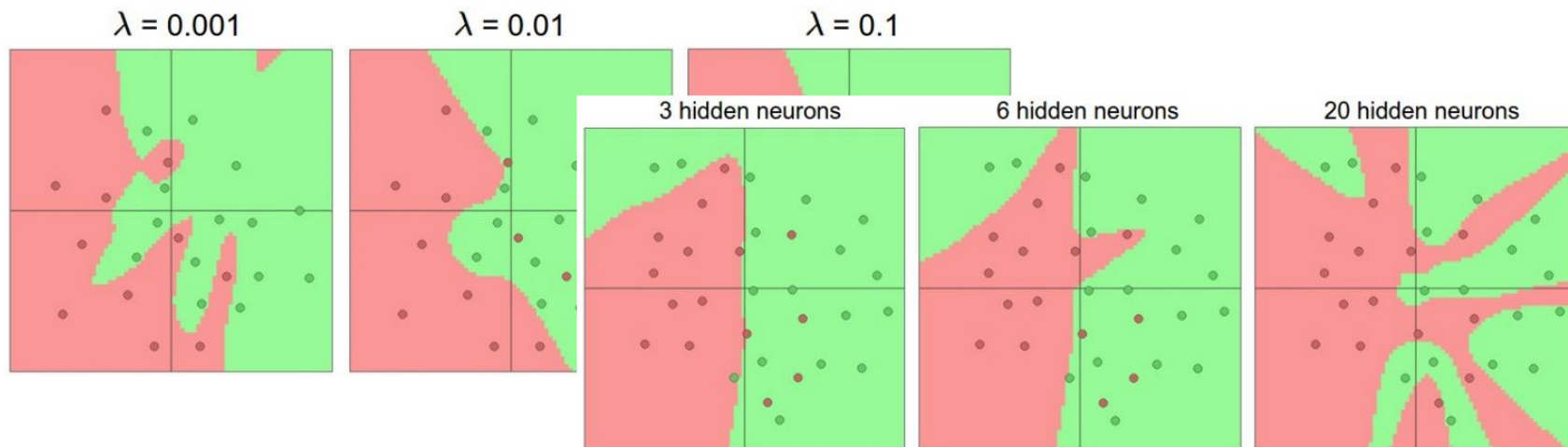


$\lambda = 0.1$



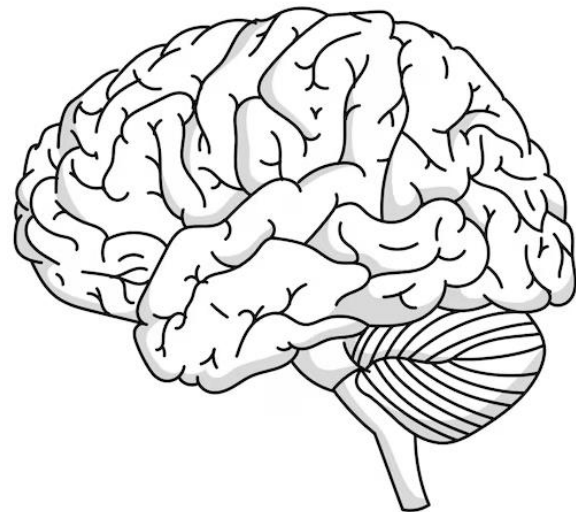
ANN (Network size and overfitting)

- Takeaways:
 - Don't be afraid of overfitting: use a network as big as you can train
 - Use regularization to control overfitting



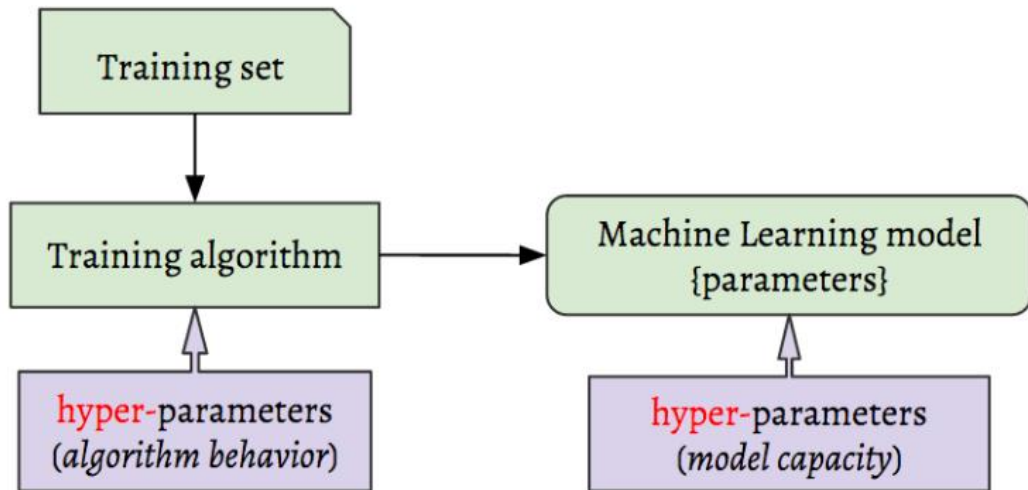
ANN (Hyperparameters)

- Parameters that are **not learned from the data** but are **set prior to training** and are used to **control** various aspects of the training process.
- These parameters are **critical** because they affect:
 - model's architecture,
 - optimization algorithm
 - training procedure
- Ultimately **influencing** the model's **performance** and **generalization** ability



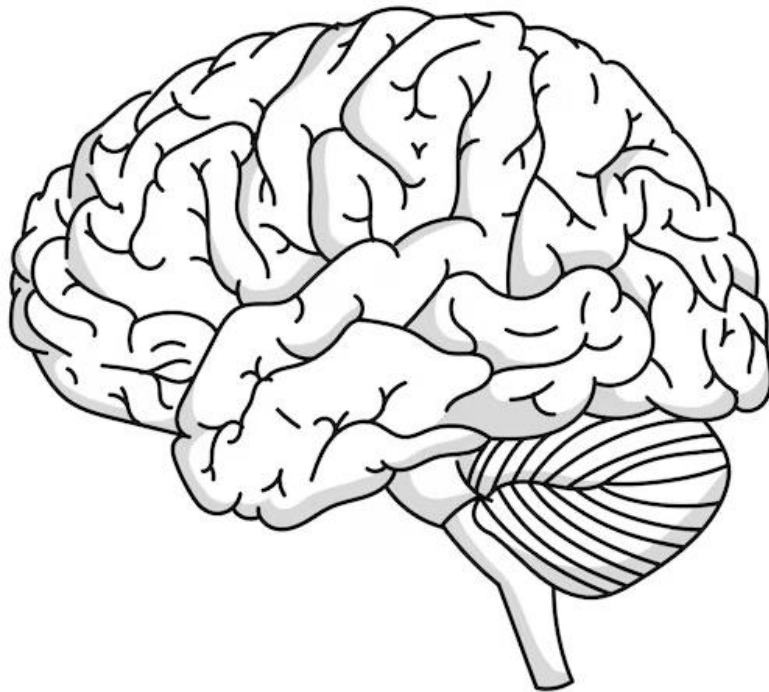
ANN (Hyperparameters)

- At every point, people always saying “it depends”



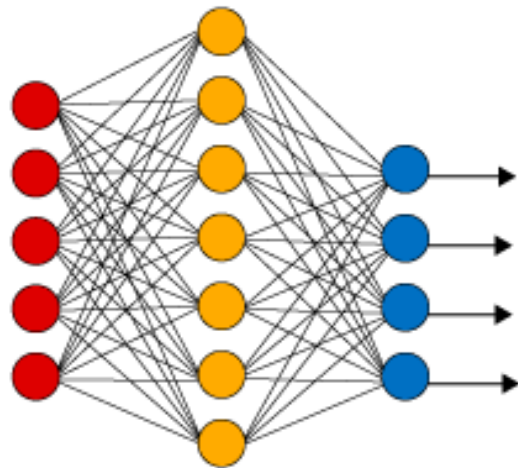
ANN (Hyperparameters)

- Some of them are:
 - Learning Rate
 - Number of Epochs
 - Batch Size
 - Architecture Hyperparameters:
 - Number of layers
 - Neurons in each layer
 - Activation functions
 - Dropout Rate:
 - Optimizers
 - Learning Rate



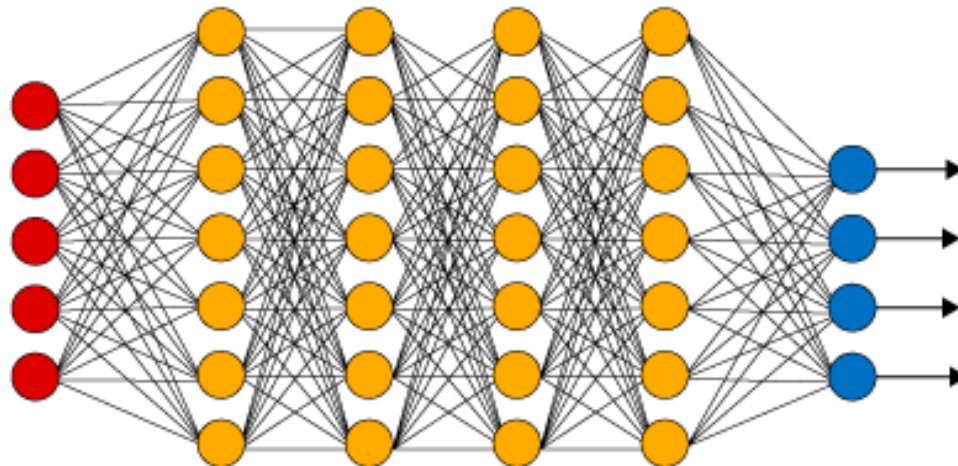
ANN (Summary)

Simple Neural Network



● Input Layer

Deep Learning Neural Network



● Hidden Layer

● Output Layer

ANN (Deep Learning Frameworks)



ANN (Code sample)

- Framework: Keras
- Less than 20 lines of code to create a simple 2-layers NN
- Simple and neat!

```
1 import tensorflow as tf
2 mnist = tf.keras.datasets.mnist
3
4 (x_train, y_train), (x_test, y_test) = mnist.load_data()
5 x_train, x_test = x_train / 255.0, x_test / 255.0
6
7 model = tf.keras.models.Sequential([
8     tf.keras.layers.Flatten(input_shape=(28, 28)),
9     tf.keras.layers.Dense(128, activation='relu'),
10    tf.keras.layers.Dropout(0.2),
11    tf.keras.layers.Dense(10, activation='softmax')
12 ])
13
14 model.compile(optimizer='adam',
15               loss='sparse_categorical_crossentropy',
16               metrics=['accuracy'])
17
18 model.fit(x_train, y_train, epochs=5)
19 model.evaluate(x_test, y_test)
```

ANN (Code sample)

- Import libraries and built-in data.
- Split data into training and testing sets.
- Normalize the training data from:
 - Pixels [0-255] -> Normalized [0-1]
- Define model
 - 1 input layer (shape of picture – 28x28)
 - 1 hidden layer (128 neurons)
 - 1 dropout layer
 - 1 output layer (10 neurons)

```
1 import tensorflow as tf
2 mnist = tf.keras.datasets.mnist
3
4 (x_train, y_train), (x_test, y_test) = mnist.load_data()
5 x_train, x_test = x_train / 255.0, x_test / 255.0
6
7 model = tf.keras.models.Sequential([
8     tf.keras.layers.Flatten(input_shape=(28, 28)),
9     tf.keras.layers.Dense(128, activation='relu'),
10    tf.keras.layers.Dropout(0.2),
11    tf.keras.layers.Dense(10, activation='softmax')
12 ])
13
14 model.compile(optimizer='adam',
15               loss='sparse_categorical_crossentropy',
16               metrics=['accuracy'])
17
18 model.fit(x_train, y_train, epochs=5)
19 model.evaluate(x_test, y_test)
```

ANN (Code sample)

- Prepare/compile the model for training:
 - Optimizer = Adaptive Momentum (Adam)
 - Loss function = Sparse categorical crossentropy
 - Metrics = Accuracy
- Train the model with 5 iterations (epochs)
- Evaluate the model on the testing set.

```
1 import tensorflow as tf
2 mnist = tf.keras.datasets.mnist
3
4 (x_train, y_train),(x_test, y_test) = mnist.load_data()
5 x_train, x_test = x_train / 255.0, x_test / 255.0
6
7 model = tf.keras.models.Sequential([
8     tf.keras.layers.Flatten(input_shape=(28, 28)),
9     tf.keras.layers.Dense(128, activation='relu'),
10    tf.keras.layers.Dropout(0.2),
11    tf.keras.layers.Dense(10, activation='softmax')
12 ])
13
14 model.compile(optimizer='adam',
15               loss='sparse_categorical_crossentropy',
16               metrics=['accuracy'])
17
18 model.fit(x_train, y_train, epochs=5)
19 model.evaluate(x_test, y_test)
```

ANN



EXERCISE 1 – Artificial Neural Network

- Packages:
 - Tensorflow
 - Scikit-learn
 - Pillow